16 · ML Platform Engineering and FinOps

> **Goal:** After this chapter you can (a) design an internal ML platform as a product with golden paths, safe multi-tenancy and measurable adoption, (b) attribute and optimise ML cost with unit economics and ranked levers, and (c) run the platform itself with SLOs, capacity plans, migrations and deprecations.

Table of Contents

- 16.1 The ML Platform as a Product: Golden Paths, Personas and Adoption
- 16.2 Multi-Tenancy, Self-Service and Policy-as-Code
- 16.3 Cost Attribution and FinOps for ML
- 16.4 Cost Optimisation Levers for Training and Inference
- 16.5 Platform SLOs, Capacity Planning, Migrations and Deprecation

16.1 The ML Platform as a Product: Golden Paths, Personas and Adoption

Conceptual Deep Dive

An ML platform is an internal product whose customers are other engineers, so it succeeds or fails on adoption, not on architecture diagrams. Start from **personas** and their jobs-to-be-done: the data scientist wants fast experiments and an easy path to production; the ML engineer wants reproducible pipelines and safe rollouts; the analyst wants self-serve batch predictions; the application developer wants a stable API contract; the auditor wants evidence without interviews. Each has different friction points; a platform serving one persona gets bypassed by the rest.

The **golden path** (a "paved road") is the default, supported way to go from idea to production: a project template wires CI, a pipeline definition, tracking, the registry, serving and monitoring together so a new model gets all of them by default. Escape hatches must exist (custom frameworks, unusual hardware) but each one has a cost the team accepts: less platform support, a manual review, no guarantee of upgrades. Too rigid a path makes teams fork; too loose a path is no path at all.

Team Topologies (Skelton and Pais, 2019) gives vocabulary: stream-aligned ML teams own models for a product area, a platform team provides a self-service internal product, and an enabling team coaches adoption of practices. Interaction modes (collaboration, X-as-a-Service, facilitation) should be explicit and time-boxed: collaboration while building a new capability, then service. A platform team that turns into a ticket queue has stopped being a platform.

Product management for platforms: roadmaps derived from user research and usage data, published internal SLAs, a deprecation policy with notice periods (16.5), and documentation treated as part of the product, with runnable examples and a "first model in a day" tutorial.

Adoption metrics show whether the product works: time to first deploy for a new project, percentage of models on the golden path, percentage with monitoring and an owner, change failure rate and lead time (DORA-style delivery measures), and user satisfaction. Counting services or features shipped measures output, not outcome.

Build, buy or adopt, layer by layer: orchestration, tracking and registry, feature store, serving, monitoring and the compute substrate. Managed offerings (SageMaker, Vertex AI, Azure ML, Databricks) bundle several layers and trade control and portability for speed and operations; open-source components (MLflow, Kubeflow, Feast and others) trade operations burden for flexibility. Decide per layer on differentiation, team size and compliance, not brand; check current docs for coverage.

Edge cases. The platform becomes a ticket queue when every change needs the platform team: invest in self-service (16.2). The golden path is so rigid that teams fork it, creating five divergent copies: add extension points. The platform ships features nobody asked for: tie each roadmap item to a named user problem and a metric.

Cross-Connection Scenarios

- Registry, promotion and approvals that the golden path wires in: Module 04 (legacy).
- Self-service, namespaces and guardrails: 16.2; cost visibility for teams: 16.3.
- Platform SLOs and deprecation mechanics: 16.5; shared GPU capacity: 9.16.
- Build-versus-buy and reference architectures in interviews: Module 12 (legacy).

Production Anti-Patterns

- Mandating the platform without making it the easiest path, so teams comply on paper and keep shadow stacks.
- Measuring platform success by the number of services or features, which rewards sprawl rather than adoption and reliability.
- No sunset policy for old workflows, so the platform accumulates legacy paths that nobody can safely remove.

Minimal Working Code

The first block computes adoption KPIs from a model inventory and a prioritised gap list.

```
import random

WEIGHTS = dict(no_owner=3, unmonitored=2, off_path=1, stale=1)

def inventory(n=40, seed=4):
    r = random.Random(seed)
    return [dict(name=f"model-{i:02d}", golden=r.random() < .35, monitored=r.random() < .
5,
                 owner=r.random() < .8, last_deploy_days=r.choice([5, 20, 60, 200, 400]))
            for i in range(n)]

def scorecard(models, stale_days=180):
    n = len(models)
    kpi = dict(golden_path=sum(m["golden"] for m in models) / n,
               monitored=sum(m["monitored"] for m in models) / n,
               owner_set=sum(m["owner"] for m in models) / n,
               stale=sum(m["last_deploy_days"] > stale_days for m in models) / n)
    gaps = sorted(((WEIGHTS["no_owner"] * (not m["owner"]) + WEIGHTS["unmonitore
d"] * (not m["monitored"])
                   + WEIGHTS["off_path"] * (not m["golden"])
                   + WEIGHTS["stale"] * (m["last_deploy_days"] > stale_days), m[&
quot;name"])
                  for m in models), reverse=True)
    return kpi, gaps

if __name__ == "__main__":
    kpi, gaps = scorecard(inventory())
    print({k: f"{v:.0%}" for k, v in kpi.items()})
    print("fix first:", gaps[:5])
```

The second block treats the golden path as code: a template dictionary is rendered into a project tree and checked for required files.

```
import tempfile
from pathlib import Path

TEMPLATE = {
    "ci/pipeline.yaml": "stages: [lint, test, train, evaluate, register]\nmodel: {name}\n&
quot;,
    "params.yaml": "seed: 42\nmodel: {name}\nmetric_gate: {gate}\n",
    "dvc.yaml": "stages:\n  train:\n    cmd: python train.py\n    deps: [data/raw]\n    ou
ts: [model.pkl]\n",
    "serving/app.py": "# serving stub for {name}\ndef predict(x):\n    raise NotImplemente
dError\n",
    "monitoring/slo.yaml": "slo: {{availability: 0.999, latency_p95_ms: {p95}}}\nowner: {o
wner}\n",
    "OWNERS": "{owner}\n",
}
REQUIRED = {"ci/pipeline.yaml", "params.yaml", "serving/app.py", "moni
toring/slo.yaml", "OWNERS"}

def scaffold(root, name, owner, gate=0.8, p95=100, template=TEMPLATE):
    base = Path(root) / name
    for rel, body in template.items():
        p = base / rel
        p.parent.mkdir(parents=True, exist_ok=True)
        p.write_text(body.format(name=name, owner=owner, gate=gate, p95=p95))
    return sorted(p.relative_to(base).as_posix() for p in base.rglob("*") if p.is_file())

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as d:
        files = scaffold(d, "churn-model", owner="team-growth")
        print("\n".join(files))
        print("golden path complete:", REQUIRED <= set(files))
```

Interview Angle

Q: You inherit 40 ad-hoc models and one platform engineer. What do you build first and how do you measure it? **A:** Inventory first: owner, last deploy, monitoring, data sources. Then one thin golden path covering the biggest pain (usually a reproducible training-to-registry pipeline with a deploy template and basic monitoring) and migrate the two or three most critical models onto it. Trade-off: depth for a few models versus shallow coverage of all. Measurable check: percentage of models with an owner and monitoring, time to first deploy for the next model, and change failure rate, tracked monthly.

Q: Platform versus embedded ML engineers: how do you decide the team structure? **A:** Embed engineers where domain speed matters and teams are few; form a platform team once several teams repeat the same infrastructure work, typically when duplicated effort exceeds the platform's staffing cost. Keep an enabling function to spread practices. Trade-off: central consistency versus local responsiveness. Measurable check: share of engineering time on undifferentiated infrastructure per team, platform adoption percentage, and satisfaction scores from a short quarterly survey.

16.2 Multi-Tenancy, Self-Service and Policy-as-Code

Conceptual Deep Dive

Many teams share one platform, so tenancy design decides how far a mistake spreads. The common model is a **namespace per team** on a shared cluster (soft multi-tenancy: shared control plane and nodes, logical isolation), with separate clusters per environment, and separate clusters or dedicated node pools for tenants with stronger isolation needs (hard multi-tenancy). The trade-off is cost and utilisation against **blast radius**: a compromised or runaway workload in a shared cluster can affect neighbours through the node, the network, the API server or shared services.

Isolation is layered. **Resource quotas and limit ranges** cap CPU, memory, GPUs and object counts per namespace and force requests and limits on pods. **Network policies** restrict traffic between namespaces. **Workload identity** gives each service account its own scoped cloud permissions instead of shared long-lived keys. Storage is separated by per-team bucket prefixes or buckets, and registries by per-team namespaces with access control. Noisy neighbours also live in shared services: a Redis instance or

GPU pool exhausted by one team's burst degrades another team's serving, so shared services need per-tenant limits or separate instances for critical paths (9.16).

Policy-as-code (OPA Gatekeeper or Kyverno at concept level; check current docs) enforces guardrails at admission time: required labels (team, owner, model), requests and limits set, images only from approved registries, no mutable latest tags, no privileged containers. Policies are versioned and tested like code, rolled out in audit mode first, then enforced. Equally important is an **exemption workflow**: time-boxed, owner-approved, logged exceptions. Policies that block an urgent fix with no escape hatch push teams to bypass the platform.

Self-service removes the ticket queue. A developer requests a project through a form or pull request; automation creates the namespace, quotas, service accounts, registry namespace and secret store paths; approvals are by exception (large GPU quota, sensitive data), not by default. The **notebook-to-pipeline path** matters for data scientists: managed notebooks with cost controls and idle shutdown, plus tooling that converts exploration into a reproducible parameterised job rather than a copy-pasted notebook.

Auditability closes the loop: who changed what (Git history and admission logs), who ran what (job metadata tied to identity), and attribution of cost and data access to teams (16.3). Audit trails also serve compliance reviews in Module 10 (legacy).

Edge cases. A noisy neighbour exhausts a shared Redis or the GPU pool: add per-tenant rate limits and quotas. A policy blocks an urgent fix and teams bypass the platform: offer fast, auditable exemptions. A single service account shared across teams destroys attribution and widens permissions: one identity per workload.

Cross-Connection Scenarios

- GPU quota and preemption policy underneath tenancy: 9.16 and 15.3.
- Admission policy and platform adoption share the golden path: 16.1.
- Quota-driven cost attribution and budgets: 16.3.
- Secrets, least privilege and data access controls: Module 10 (legacy).

Production Anti-Patterns

- Everyone cluster-admin, so any mistake or compromise has cluster-wide blast radius and nothing can be attributed.
- Policies without exemption workflows, which teach engineers to route around the platform.
- Manual onboarding via tickets, so new teams wait days and the platform team becomes the bottleneck.

Minimal Working Code

The first block is an admission-style evaluator over a pod-like manifest, with time-boxed exemptions and tests on good and bad inputs.

```
APPROVED = ("registry.example.test/",)
REQUIRED_LABELS = ("team", "owner", "model")

def evaluate(manifest, exemptions=None, today="2026-01-01"):
    """Return violations as (rule, message); exemptions: {rule: 'YYYY-MM-DD' expiry}."
;""
    v = []
    labels = manifest.get("metadata", {}).get("labels", {})
    v += [("labels", f"missing label {k}") for k in REQUIRED_LABELS if k not in label
s]
    for c in manifest.get("spec", {}).get("containers", []):
        img, name = c.get("image", ""), c.get("name", "?")
        if not img.startswith(APPROVED): v.append(("registry", f"{name}: unapproved regist
ry"))
        last = img.rsplit("/", 1)[-1]
        if "@sha256:" not in img and (":" not in last or last.endswith(":latest&
quot;)):
            v.append(("latest-tag", f"{name}: mutable or missing tag"))
        res = c.get("resources", {})
        if not (res.get("requests") and res.get("limits")):
            v.append(("resources", f"{name}: requests and limits required"))
    exemptions = exemptions or {}
    return [(r, m) for r, m in v if exemptions.get(r, "") < today]

GOOD = {"metadata": {"labels": {"team": "a", "owner": &
```

```
quot;x", "model": "m"}},
        "spec": {"containers": [{"name": "app", "image"
: "registry.example.test/m:1.2.0",
                                "resources": {"requests": {"cpu": "1&
quot;}, "limits": {"cpu": "2"}}}]}}
if __name__ == "__main__":
    assert evaluate(GOOD) == []
    bad = {"metadata": {"labels": {"team": "a"}}, "spec"
:: {"containers": [{"name": "app", "image": "docker.io/m:lat
est"}]}}
    for rule, msg in evaluate(bad): print(rule, "->", msg)
    print("with exemption:", [r for r, _ in evaluate(bad, {"latest-tag": "2026-1
2-31"}, today="2026-06-01")])
```

The second block computes per-team quotas from requests and historical usage with a headroom parameter and an explanation for each decision.

```
import math

def allocate(teams, capacity, headroom=0.25, floor=1):
    """teams: {name: dict(requested=int GPUs, p95_used=float GPUs)} -> quotas, explanat
ions, spare."""
    want = {t: max(floor, min(d["requested"], math.ceil(d["p95_used"] * (1 + headroom
))))
            for t, d in teams.items()}
    scale = min(1.0, capacity / sum(want.values()))
    quota = {t: max(floor, math.floor(w * scale)) for t, w in want.items()}
    notes = {}
    for t, d in teams.items():
        why = []
        if want[t] < d["requested"]:
            why.append(f"capped at p95 x (1+{headroom:.0%}) = {want[t]}, requested {d['requested']}&
quot;)
        if scale < 1:
            why.append(f"scaled by {scale:.2f}: demand exceeds capacity")
        notes[t] = "; ".join(why) or "granted as requested"
    return quota, notes, capacity - sum(quota.values())

if __name__ == "__main__":
    teams = {"vision": dict(requested=24, p95_used=14.2),
             "ranking": dict(requested=16, p95_used=15.0),
             "research": dict(requested=32, p95_used=9.5)}
    quota, notes, spare = allocate(teams, capacity=48)
    for t in teams: print(f"{t:9s} quota={quota[t]:2d}  {notes[t]}")
    print("spare for borrowing:", spare)
```

Interview Angle

Q: Design self-service for 50 data scientists with safe defaults and escape hatches. **A:** A pull-request or portal flow that creates a namespace, quotas, service account, registry namespace and storage prefix from a template, with admission policies in audit-then-enforce mode and managed notebooks with idle shutdown. Approvals only by exception (large quotas, sensitive data); exemptions are time-boxed and logged. Trade-off: guardrails slow unusual work, so make the exemption path fast. Measurable check: time from request to first job, percentage of requests needing human approval, policy violations per week.

Q: How do you stop one team's training burst from degrading another team's serving? **A:** Separate node pools or priority classes so serving preempts batch, namespace quotas and limit ranges, network policies, per-tenant limits on shared services (cache, feature store), and autoscaling headroom for serving. Trade-off: reserved serving capacity idles at night, offset by preemptible batch backfill. Measurable check: serving p99 and error rate during synthetic training bursts in a game-day test, and preemption counts for batch work.

16.3 Cost Attribution and FinOps for ML

Conceptual Deep Dive

Cost decisions need **unit economics**: cost per prediction, per 1,000 predictions, per training run, per experiment and per active model, and where possible revenue or value per prediction so cost can be judged against benefit. Total spend is a poor steering metric: a doubling can be healthy growth or waste, and only a unit metric tells which.

Allocation starts with tagging or labelling every resource with team, model, environment and cost centre, enforced by policy at creation (16.2). Shared costs (clusters, GPUs, platform services, observability) need a rule: allocate by measured use (GPU-hours actually used), by request, or evenly. Allocating by request rewards over-requesting and starts disputes; allocating by usage is fairer but needs metering. **Showback** reports cost to teams for awareness; **chargeback** bills it to their budgets and changes behaviour more strongly but needs trusted numbers.

The **cost drivers** in ML are GPU hours (training, HPO sweeps, idle notebooks), always-on serving replicas, storage and egress (9.17), feature-store and online-store capacity, and logging and monitoring volume (8.18). The expensive surprises are usually idle or forgotten resources and runaway experiments, not the headline training run.

Visibility means dashboards by team, model and environment, anomaly alerts on daily spend, and budgets with forecasts and thresholds (for example alert at 80% of a monthly budget). The **FinOps** cycle (inform, optimise, operate) repeats: make costs visible and attributed, pull optimisation levers (16.4), then embed review into regular operations, with cost reviews alongside reliability reviews. Treat cost as another SLO: a budget per model, and guardrails such as maximum trial counts and GPU-hour caps on HPO sweeps and backfills (15.4).

Edge cases. Untagged resources form an "unallocated" bucket that grows; track its share as a KPI and fail resource creation without tags. Allocating shared clusters by request versus usage causes disputes: publish the method and show both numbers. Experiment sprawl, thousands of small runs and orphaned artifacts, accumulates storage and compute: apply retention rules with registry-aware safety (9.17).

Cross-Connection Scenarios

- Cluster utilisation versus allocation underpins fair cost splits: 9.16 and 15.3.
- Tag enforcement and per-team quotas as policy: 16.2.
- Levers that change the unit costs measured here: 16.4.
- Logging volume and metric cardinality as cost drivers: 8.18.

Production Anti-Patterns

- Optimising cost without measuring utilisation, so savings plans lock in capacity that sits idle.
- Looking only at the monthly invoice, which arrives too late to prevent a runaway job or explain a spike.
- No budget guardrail on sweeps and backfills, so one misconfigured experiment consumes a quarter's allowance.

Minimal Working Code

The first block splits shared cluster cost by GPU-hours used rather than requested, reports idle cost and the untagged share, and computes cost per 1,000 predictions.

```
def allocate(total_cost, capacity_gpu_h, teams):
    """Costs are abstract units. teams: {name: dict(used=gpu_h, requested=gpu_h)}."&q
uot;"
    rate = total_cost / capacity_gpu_h                    # cost per provisioned GPU-hour
    used_total = sum(t["used"] for t in teams.values())
    idle = total_cost - used_total * rate
    out = {}
    for name, t in teams.items():
        direct = t["used"] * rate
        out[name] = dict(direct=round(direct, 1),
                         by_request=round(t["requested"] * rate, 1),
                         with_idle_share=round(direct + idle * t["used"] / used_total, 1))
    return out, round(idle, 1)

def per_1k(cost, predictions):
    return 1000 * cost / predictions

if __name__ == "__main__":
```

```
    teams = {"vision": dict(used=3000, requested=5000), "ranking": dict(used=4200, re
quested=4500),
             "untagged": dict(used=800, requested=800)}
    out, idle = allocate(total_cost=100_000, capacity_gpu_h=12_000, teams=teams)
    for n, row in out.items(): print(f"{n:9s}", row)
    print("idle (provisioned but unused):", idle, "| untagged share of used:",
          f"{teams['untagged']['used'] / sum(t['used'] for t in teams.values()):.1%}")
    print("cost per 1k predictions, ranking model:", round(per_1k(out['ranking']['direct'], 90_
000_000), 4))
```

The second block detects spend anomalies with a rolling median and MAD (modified z-score) and names the top contributing tag.

```
import numpy as np

def mad_z(hist, x):
    med = np.median(hist)
    mad = np.median(np.abs(hist - med)) or 1e-9
    return 0.6745 * (x - med) / mad

def detect(daily, by_tag, window=14, thresh=5.0):
    alerts = []
    for d in range(window, len(daily)):
        hist = np.array(daily[d - window:d])
        z = mad_z(hist, daily[d])
        if z > thresh:
            delta = {t: s[d] - np.median(s[d - window:d]) for t, s in by_tag.items()}
            top = max(delta, key=delta.get)
            alerts.append(f"day {d}: spend {daily[d]:.0f} vs median {np.median(hist):.0f} "
                          f"(robust z={z:.1f}); top tag: {top} (+{delta[top]:.0f})")
    return alerts

if __name__ == "__main__":
    rng = np.random.default_rng(0)
    days = 30
    by_tag = {"serving": 100 + rng.normal(0, 3, days), "training": 80 + rng.normal(0,
 5, days),
              "sweep=hpo-17": 10 + rng.normal(0, 1, days)}
    by_tag["sweep=hpo-17"][25:27] += 120                    # a runaway sweep
    daily = sum(by_tag.values()).tolist()
    for a in detect(daily, by_tag): print(a)
```

Interview Angle

Q: Inference spend doubled in a quarter. How do you investigate before changing anything? **A:** Decompose the change: unit cost (cost per 1,000 predictions) versus volume, by model, team, environment and tag. Check replica counts and autoscaler behaviour, hardware type changes, traffic growth, retry storms, new models in shadow mode, logging volume and egress. Compare against the deploy and incident timeline. Trade-off: attribution quality limits precision; untagged spend blurs the answer. Measurable check: the split of the increase into volume, unit cost and new workloads accounts for nearly all of the delta, with the unallocated share small.

Q: Define cost KPIs for an ML platform and who sees them. **A:** Cost per 1,000 predictions per model (model owners), GPU idle share and queue wait (platform team), cost per training run and per experiment (ML teams), unallocated percentage (finance and platform), and budget burn versus forecast (team leads). Trade-off: more KPIs raise noise; keep each with a named owner and an action threshold. Measurable check: every KPI has an owner, a target and a monthly review, and the unallocated share trends below an agreed bound.

16.4 Cost Optimisation Levers for Training and Inference

Conceptual Deep Dive

Cost work is a ranking problem: many levers exist, each with an expected saving, a risk to quality or reliability, and an engineering effort. Rank by saving per unit of risk-adjusted effort, apply the cheap safe ones first, and verify each with a measurement, never

an assumption.

Training levers. Spot or preemptible capacity with checkpointing (15.2); right-sized instances chosen after profiling (15.5); mixed precision; early stopping and multi-fidelity HPO (15.4); smaller proxy models or data subsets for search; caching expensive features and preprocessed data; scheduling flexible jobs off-peak; and killing idle notebooks and orphaned runs.

Inference levers. Dynamic batching raises accelerator utilisation; quantisation and distillation shrink models (accuracy must be re-validated on slices); cascades route easy requests to a cheap model and hard ones to an expensive one; caching serves repeated inputs (bounded by a staleness tolerance); autoscaling on leading indicators (queue depth, concurrency) with tuned scale-down avoids both idle replicas and cold-start pain; and right-sizing decides between CPU and GPU per model (Module 06 (legacy)). **Batch versus online**: if freshness allows, precomputing predictions turns an always-on GPU service into a nightly job.

Storage and data levers. Tiering cold data, lifecycle rules (only with registry-aware safety, 9.17), downsampling and sampling logs (8.18), columnar formats, and deleting unreferenced intermediate artifacts.

Commitments. Reserved or committed-use purchases lower the unit cost of steady load, spot suits interruptible work, on-demand covers uncertainty. Commit only to the stable base that utilisation data supports; buying before usage stabilises converts a variable cost into an idle one. Prices are provider specific and change; keep them as parameters and check current provider documentation.

Levers **compose with diminishing returns**: if quantisation cuts inference cost by a fraction, batching then applies to the remaining cost, so savings multiply on the remainder rather than adding. Always keep quality guardrails (accuracy on slices, latency percentiles, error rate) as release criteria for each lever, because a saving that degrades the product is a loss.

Edge cases. Savings realised on paper but utilisation unchanged: a smaller instance bought while the old one still runs, or a cheaper rate on idle capacity. Spot interruptions can cost more than they save when checkpoints are infrequent or restarts slow (the simulation below finds the break-even). Caching beyond the staleness tolerance returns wrong answers cheaply: define maximum age per use case and monitor hit rate and error.

Cross-Connection Scenarios

- Spot-safe training depends on checkpoint and resume behaviour from 15.2.
- Measuring utilisation and throughput per dollar before choosing hardware: 15.5.
- Cost visibility to confirm that savings landed: 16.3.
- Dynamic batching, quantisation and caching mechanics: Module 06 (legacy).

Production Anti-Patterns

- Moving to cheaper hardware without profiling, so a memory- or bandwidth-bound workload slows down and the saving vanishes in extra replicas.
- Committing capacity before usage stabilises, locking in a base that later sits idle.
- Cutting monitoring and logging to save money, which removes the visibility needed to detect the quality damage other cuts may cause.

Minimal Working Code

The first block ranks levers from assumed savings, risk and effort, and composes savings multiplicatively per cost category.

```
def plan(baseline, levers):
    """baseline: {category: cost units}. levers: {name: (category, saving_fraction, risk 0
-1, effort 1-5)}.
    All numbers are assumptions to be replaced with measurements."""
    score = lambda kv: baseline[kv[1][0]] * kv[1][1] / (kv[1][3] * (1 + kv[1][2]))
    remaining, steps = dict(baseline), []
    for name, (cat, s, risk, effort) in sorted(levers.items(), key=score, reverse=True):
        saved = remaining[cat] * s                # diminishing returns: applies to what is left
        remaining[cat] -= saved
        steps.append((name, round(saved, 1), f"risk={risk}", f"effort={effort}"))
    total = sum(baseline.values())
    return steps, total - sum(remaining.values()), total

if __name__ == "__main__":
    baseline = dict(training=40, inference=45, storage=10, observability=5)
    levers = {"spot + checkpoints": ("training", .30, .3, 3), "mixed precision&q
```

```
uot;: ("training", .15, .2, 2),
            "dynamic batching": ("inference", .25, .2, 2), "quantisation"
: ("inference", .30, .5, 4),
            "response cache": ("inference", .15, .4, 2), "autoscaler tuning&qu
ot;: ("inference", .15, .1, 1),
            "storage lifecycle": ("storage", .30, .3, 1), "log downsampling&qu
ot;: ("observability", .40, .3, 1)}
    steps, saved, total = plan(baseline, levers)
    for s in steps: print(s)
    naive = sum(baseline[c] * s for c, s, _, _ in levers.values())
    print(f"combined saving {saved:.1f}/{total} ({saved / total:.0%}) vs naive additive {naive:.1f}&
quot;)
```

The second block simulates spot versus on-demand cost with random preemptions, checkpoint overhead and restart delay, and shows where spot stops paying.

```
import numpy as np

def spot_cost(work_h, lam, ckpt_every, ckpt_cost, delay, price, rng):
    """Wall-clock cost of finishing work_h on spot; preemptions arrive at rate lam per hou
r."""
    done = wall = 0.0
    while done < work_h:
        chunk = min(ckpt_every, work_h - done)
        need = chunk + ckpt_cost                # run one checkpoint interval, then save
        t = rng.exponential(1 / lam)            # time to next preemption
        if t >= need:
            wall += need; done += chunk
        else:
            wall += t + delay                   # lose partial chunk, wait for replacement
    return wall * price

if __name__ == "__main__":
    work, o, delay, price = 100, 0.02, 0.25, 0.4   # price: spot as a fraction of on-demand (assumed)
    print("preemptions/hour:", [0.1, 0.2, 0.4, 0.8])
    for c in (0.25, 1.0, 4.0):
        on_demand = work * (1 + o / c)          # same checkpoint overhead, never preempted
        ratios = []
        for lam in (0.1, 0.2, 0.4, 0.8):
            rng = np.random.default_rng(0)
            mean = np.mean([spot_cost(work, lam, c, o, delay, price, rng) for _ in range(100)])
            ratios.append(round(float(mean / on_demand), 2))
        print(f"checkpoint every {c:4.2f}h -> spot/on-demand cost ratio {ratios}  (>1 means sp
ot loses)")
```

Interview Angle

Q: You must cut inference cost 50% without losing quality. What is your plan and order? **A:** Measure first: cost per 1,000 predictions, utilisation, and the traffic mix. Then the safest levers: autoscaler and replica right-sizing, dynamic batching, response caching for repeats, and batch precompute where freshness allows; next quantisation or a model cascade with slice-level quality checks. Trade-off: each lever adds risk or complexity. Measurable check: per-lever shadow or canary showing quality within a tolerance (for example accuracy and p99 latency guardrails) and realised unit cost, not projected cost.

Q: When does spot make training more expensive overall? **A:** When preemptions are frequent relative to the checkpoint interval and restart delay, so wasted work and waiting exceed the price discount, or when jobs cannot checkpoint, or when gang restarts of many workers magnify each loss. Trade-off: shorter checkpoints cost overhead but cut waste. Measurable check: simulate or measure effective cost as wall-clock hours times price versus on-demand, and track preemption rate and re-run fraction per job.

16.5 Platform SLOs, Capacity Planning, Migrations and Deprecation

Conceptual Deep Dive

The platform needs its own reliability targets. Candidate **platform SLOs**: pipeline success rate (excluding user code errors), GPU queue wait percentile by job class, registry and metadata-store availability, deploy lead time from merge to serving, and incident time to recover. The mechanics follow 8.17: define good-event ratios, set a budget, alert on burn rate, and let the budget steer prioritisation between new features and reliability. Users trust a platform that publishes what it promises and reports against it.

Capacity planning forecasts GPUs and storage from usage history (trend plus seasonality, as in release cycles and quarter ends), adds headroom proportional to the wait-time target, and accounts for **lead times**: if procurement or reservations take months, the forecast horizon must exceed lead time or shortages are unavoidable. Use scenarios (base, high growth, a big new model) rather than a single line, and re-forecast regularly (15.3 covers the demand-percentile view).

Migrations (between serving stacks, feature stores, registries) use the strangler pattern: route a growing share of traffic or models to the new system while the old one remains authoritative. **Dual-running** with replayed or mirrored traffic gives parity checks: agreement rates, maximum differences and slice-level mismatches, with an explicit go or no-go rule. Cut over per model, keep a tested rollback, and decommission only after a soak period. Parity gaps often appear only on rare slices, so aggregate agreement is not enough.

Deprecation is a process: inventory consumers, notify owners with dates, collect usage telemetry to see who still calls the old path, escalate with a hard deadline, and verify removal (zero traffic, zero references). A long tail of consumers that refuses to migrate needs executive-level deadline backing or active migration help from the platform team.

Platform reliability engineering reduces blast radius: tenant isolation (16.2), cell-based or per-team deployments for critical services, load shedding, dependency timeouts, and chaos or game-day drills that rehearse failures (registry down, GPU pool lost, feature store lag). Communicate health to leadership in business terms: adoption, reliability against published SLOs, cost per unit (16.3) and migration progress, not infrastructure counters.

Edge cases. Migration parity gaps appear only on rare slices, hence slice-level checks with minimum support. A long tail of consumers blocks removal of the old path. Capacity lead time exceeding the forecast horizon makes every spike an incident.

Cross-Connection Scenarios

- Burn-rate alerting and error budgets applied to platform SLOs: 8.17.
- Queue wait and utilisation inputs to capacity plans: 15.3 and 9.16.
- Backup, restore and DR for the platform's registry and stores: 9.18.
- Registry-driven rollouts and rollback discipline: Module 04 and Module 07 (legacy).

Production Anti-Patterns

- A big-bang platform migration with no dual-running, so any parity gap becomes an outage for every model at once.
- Deprecating without telemetry on actual usage, which makes removal a guess and surprises hidden consumers.
- No rollback plan for platform upgrades, so the platform team cannot undo a bad release under pressure.

Minimal Working Code

The first block checks migration parity between an old and a new system on replayed traffic, including slices, and applies a go or no-go rule.

```
import numpy as np

def parity(old, new, slices, tol=1e-3, min_n=50, overall_min=0.995, slice_min=0.98, max_abs=0.05):
    close = np.abs(old - new) <= tol
    rep = dict(agreement=round(float(close.mean()), 4), max_diff=round(float(np.abs(old - new).max()), 3)
,
               bad_slices={})
    for s in np.unique(slices):
        m = slices == s
        if m.sum() >= min_n and close[m].mean() < slice_min:
            rep["bad_slices"][str(s)] = round(float(close[m].mean()), 3)
    rep["go"] = rep["agreement"] >= overall_min and not rep["bad_slices"
] and rep["max_diff"] <= max_abs
    return rep

if __name__ == "__main__":
    rng = np.random.default_rng(0)
    n = 20000
```

```
    old = rng.random(n)
    new = old + rng.normal(0, 1e-5, n)                    # numerically equivalent system
    slices = np.where(rng.random(n) < 0.005, "rare", "common")
    bug = (slices == "rare") & (rng.random(n) < 0.6)      # new stack mishandles a rare
 slice
    new[bug] += 0.04
    print(parity(old, new, slices))
```

The second block forecasts monthly GPU needs from a linear trend and seasonal factors, with headroom and procurement lead-time warnings.

```
import numpy as np

def forecast(history, seasonal, horizon, headroom=0.2, capacity=None, lead_months=3):
    """history: monthly peak GPU demand (oldest first); seasonal: 12 factors averaging 1.&
quot;""
    n = len(history)
    base = np.array([h / seasonal[i % 12] for i, h in enumerate(history)])
    slope, intercept = np.polyfit(np.arange(n), base, 1)
    rows, warns = [], []
    for k in range(1, horizon + 1):
        m = n - 1 + k
        need = (intercept + slope * m) * seasonal[m % 12] * (1 + headroom)
        rows.append((f"+{k}m", round(float(need))))
        if capacity is not None and need > capacity:
            when = "ALREADY LATE" if k <= lead_months else f"order by +{k - lead_months
}m"
            warns.append(f"+{k}m: need {need:.0f} > capacity {capacity}: {when} (lead time {lead_
months}m)")
    return rows, warns

if __name__ == "__main__":
    seasonal = [0.95, 0.95, 1.0, 1.0, 1.0, 1.0, 0.9, 0.9, 1.0, 1.05, 1.15, 1.1]
    rng = np.random.default_rng(0)
    history = [(40 + 2 * i) * seasonal[i % 12] + rng.normal(0, 0.5) for i in range(12)]
    rows, warns = forecast(history, seasonal, horizon=6, capacity=75)
    print(rows)
    print("\n".join(warns))
```

Interview Angle

Q: Describe how you'd migrate 30 models from a legacy serving stack with zero downtime. **A:** Strangler pattern: stand up the new stack, mirror or replay production traffic to it, and run parity checks per model (agreement rate, max difference, slice-level mismatches, latency). Cut over models in waves, lowest risk first, via weighted routing with instant rollback, keep the old path hot through a soak period, then decommission with telemetry confirming zero traffic. Trade-off: dual-running doubles cost temporarily. Measurable check: parity thresholds per model, error rate and p99 during each wave, rollback drills.

Q: Which SLOs would you publish for an internal ML platform? **A:** Pipeline success rate excluding user-code failures, queue wait percentiles per job class (for example p90 for interactive and batch), registry and metadata availability, deploy lead time, and time to restore after incidents, each with a window and error budget. Trade-off: tight SLOs demand spare capacity and on-call investment. Measurable check: burn-rate alerts tied to each SLO, a monthly report against targets, and budget exhaustion triggering a reliability-first sprint.